

Учёт трудозатрат и стоимость работ по проектам

Fullstack-разработчик: .NET, MongoDB, React + TypeScript

ОБЪЁМ	СРОК	ЧАСТЕЙ	ФОРМАТ СДАЧИ
6–8 часов	5 календарных дней	2 + разбор	ссылка на репозиторий

О проекте

ERP-система для проектной организации: договоры и проекты, сотрудники, табель рабочего времени, зарплата и премии, накладные расходы, рентабельность, склад, документооборот. Система работает в продакшене несколько лет, база живая, пользователи реальные.

Backend	.NET 5 (C#), REST + SignalR, CQRS через MediatR, FluentValidation
База	MongoDB — единственное хранилище, реляционной БД нет: агрегации, change streams
Frontend	React 16 + TypeScript, mobx-state-tree, Formik + Yup, ag-Grid, Blueprint.js
Прочее	Docker, Hangfire, генерация документов из шаблонов

Работа примерно на 80 % — доработка и починка существующего кода, на 20 % — новые модули. Поэтому задание состоит из двух частей: прочитать чужой код и написать свой.

Правила

- **Ориентировочное время — 6–8 часов.** Не нужно вылизывать: нужно, чтобы работало и было понятно, почему сделано именно так.

- **Вопросы приветствуются.** Спецификация намеренно оставляет несколько мест недосказанными. Уточняющий вопрос — это плюс к оценке, а не минус. Решили не спрашивать — зафиксируйте допущение в `NOTES.md`.
- **ИИ-ассистенты разрешены** — Claude Code, Copilot, Cursor: мы сами ими пользуемся. Условие одно: на разборе вы объясняете любую строку и говорите, где вы с ассистентом не согласились.
- **Дизайн не оценивается.** Оцениваются корректность, структура и обработка ошибок.

ЧАСТЬ 1 • 1–1,5 ЧАСА

Code review

Ниже два файла из учебного проекта, написанные «как получилось». Они компилируются и в happy path работают. Задача — **найти проблемы и отранжировать их по важности**.

Оформите ответ в файле `REVIEW.md`:

1. Список найденных проблем: файл и фрагмент, в чём суть, **чем это грозит в продакшене**, как чинить. Сортировка — от самой опасной к косметике.
2. Отдельно: что бы вы изменили в *структуре* этого кода, а не в отдельных строках.
3. Одну-две проблемы, которые считаете самыми важными, исправьте прямо в коде — положите исправленные файлы рядом, например `TimesheetReportHandler.fixed.cs`.

`TimesheetReportHandler.cs` — отчёт «стоимость трудозатрат по проектам за месяц»

```
// Учебный проект. Обработчик отчёта "стоимость трудозатрат по проектам за месяц".
// Код рабочий: на небольшой базе отчёт строится и цифры выглядят правдоподобно.

using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading;
using System.Threading.Tasks;
using MediatR;
using MongoDB.Driver;

namespace Demo.Api.Queries.Reports
{
    public class ProjectReportRow
    {
        public string ProjectId { get; set; }
        public string ProjectName { get; set; }
    }
}
```

```

        public double Hours { get; set; }
        public double Amount { get; set; }
        public double Budget { get; set; }
        public double Percent { get; set; }
        public bool Overspent { get; set; }
    }

    public class GetProjectReportQuery : IRequest<List<ProjectReportRow>>
    {
        public int Year { get; set; }
        public int Month { get; set; }
    }

    public class TimesheetReportHandler : IRequestHandler<GetProjectReportQuery,
List<ProjectReportRow>>
    {
        private readonly IMongoDatabase _db;

        public TimesheetReportHandler(IMongoDatabase db)
        {
            _db = db;
        }

        public async Task<List<ProjectReportRow>> Handle(GetProjectReportQuery request,
Cancellation token)
        {
            var entries = await _db.GetCollection<TimeEntry>("time_entries")
                .Find(FilterDefinition<TimeEntry>.Empty)
                .ToListAsync();

            var monthEntries = entries
                .Where(e => e.Date.Year == request.Year && e.Date.Month == request.Month)
                .ToList();

            var rows = new Dictionary<string, ProjectReportRow>();

            foreach (var entry in monthEntries)
            {
                var employee = _db.GetCollection<Employee>("employees")
                    .Find(e => e.Id == entry.EmployeeId)
                    .FirstOrDefaultAsync().Result;

                var rate = employee.Rates.FirstOrDefault().Value;

                var amount = Math.Round(entry.Hours * rate, 2);

                if (!rows.ContainsKey(entry.ProjectId))
                {
                    var project = await _db.GetCollection<Project>("projects")
                        .Find(p => p.Id == entry.ProjectId)
                        .FirstOrDefaultAsync();

                    rows[entry.ProjectId] = new ProjectReportRow
                    {
                        ProjectId = project.Id,
                        ProjectName = project.Name,
                        Budget = project.Budget
                    };
                }
            }
        }
    }

```

```

        rows[entry.ProjectId].Hours += entry.Hours;
        rows[entry.ProjectId].Amount += amount;
    }

    foreach (var row in rows.Values)
    {
        row.Percent = Math.Round(row.Amount / row.Budget * 100, 2);
        row.Overspent = row.Percent > 100;
    }

    return rows.Values.OrderBy(r => r.ProjectName).ToList();
}

// --- сущности (упрощённо) ---

public class TimeEntry
{
    public string Id { get; set; }
    public string EmployeeId { get; set; }
    public string ProjectId { get; set; }
    public DateTime Date { get; set; }
    public double Hours { get; set; }
    public string Comment { get; set; }
}

public class Employee
{
    public string Id { get; set; }
    public string Name { get; set; }
    public List<Rate> Rates { get; set; }
}

public class Rate
{
    public DateTime From { get; set; }
    public double Value { get; set; }
}

public class Project
{
    public string Id { get; set; }
    public string Name { get; set; }
    public double Budget { get; set; }
}
}

```

TimeEntriesPage.tsx – экран «Табель»

```

// Учебный проект. Экран "Табель": список записей за месяц с фильтрами и добавлением.
// Код рабочий: записи отображаются, добавление работает.

import React, { useState, useEffect } from "react";

interface Props {

```

```

    year: number;
    month: number;
  }

export const TimeEntriesPage = (props: Props) => {
  const [entries, setEntries] = useState<any[]>([]);
  const [employees, setEmployees] = useState<any[]>([]);
  const [employeeId, setEmployeeId] = useState("");
  const [hours, setHours] = useState("");
  const [date, setDate] = useState("");
  const [projectId, setProjectId] = useState("");
  const [loading, setLoading] = useState(false);

  useEffect(() => {
    load();
  });

  useEffect(() => {
    fetch("/api/employees")
      .then((r) => r.json())
      .then((data) => setEmployees(data));
  }, []);

  const load = async () => {
    setLoading(true);
    const response = await fetch("/api/time-entries?year=" + props.year + "&month=" +
props.month);
    const data = await response.json();
    setEntries(data);
    setLoading(false);
  };

  const filtered = employeeId ? entries.filter((e) => e.employeeId == employeeId) : entries;

  let total = 0;
  for (let i = 0; i < filtered.length; i++) {
    total = total + parseFloat(filtered[i].amount);
  }

  const save = async () => {
    const body = {
      employeeId: employeeId,
      projectId: projectId,
      date: new Date(date).toLocaleDateString(),
      hours: hours,
    };

    await fetch("/api/time-entries", {
      method: "PUT",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(body),
    });

    entries.push(body);
    setEntries(entries);
    alert("Сохраниено");
  };

  const remove = async (id: string) => {

```

```

    await fetch("/api/time-entries/" + id, { method: "DELETE" });
    load();
  };

  return (
    <div style={{ padding: 20 }}>
      <h2>Табель за {props.month}.{props.year}</h2>

      <select value={employeeId} onChange={(e) => setEmployeeId(e.target.value)}>
        <option value="">Все сотрудники</option>
        {employees.map((emp, index) => (
          <option key={index} value={emp.id}>
            {emp.name}
          </option>
        ))}
      </select>

      <div style={{ marginTop: 20 }}>
        <input placeholder="Дата" value={date} onChange={(e) => setDate(e.target.value)}
      />
        <input placeholder="Проект" value={projectId} onChange={(e) =>
setProjectId(e.target.value)} />
        <input placeholder="Часы" value={hours} onChange={(e) =>
setHours(e.target.value)} />
        <button onClick={save}>Добавить</button>
      </div>

      {loading && <div>Загрузка...</div>}

      <table style={{ marginTop: 20, width: "100%" }}>
        <tbody>
          {filtered.map((entry, index) => (
            <tr key={index}>
              <td>{entry.date}</td>
              <td>{entry.employeeName}</td>
              <td>{entry.projectName}</td>
              <td>{entry.hours}</td>
              <td>{entry.amount.toFixed(2)}</td>
              <td>
                <button onClick={() => remove(entry.id)}>Удалить</button>
              </td>
            </tr>
          ))}
        </tbody>
      </table>

      <div style={{ marginTop: 10 }}>Итого: {total.toFixed(2)} руб.</div>
    </div>
  );
};

```

Мини-фича end-to-end

Отдельный маленький проект на том же стеке — **учёт трудозатрат и стоимость работ по проектам**.

Домен

сущность	поля
Employee сотрудник	ФИО, отдел; история часовых ставок — список пар «ставка, действует с даты». Ставка действует с указанной даты до начала следующей. Ставки задаются заранее, меняются редко, но задним числом их менять можно .
Project проект	шифр (уникальный, например П-001), название, бюджет в рублях, дата начала и дата окончания. Окончание может отсутствовать — проект бессрочный.
TimeEntry запись табеля	сотрудник, проект, дата, количество часов, комментарий; служебные поля на ваше усмотрение — кто и когда создал или изменил, версия записи.
ClosedPeriod закрытый период	год и месяц, закрытые для редактирования: их закрывает бухгалтерия.

Бизнес-правила

1. **Стоимость записи = часы × ставка сотрудника, действовавшая на дату записи** — не текущая. Если на дату записи у сотрудника ещё нет ни одной ставки, запись создать нельзя.
2. Суммарно у сотрудника **за один календарный день по всем проектам не больше 24 часов**. Попытка выйти за лимит — ошибка с внятным текстом.
3. Если за день у сотрудника получилось **больше 12 часов**, запись сохраняется, но день помечается как переработка, и флаг виден в интерфейсе.
4. В **закрытом периоде** записи нельзя создавать, изменять и удалять.
5. Дата записи должна попадать в **период проекта**: не раньше даты начала и не позже даты окончания, если она задана.
6. Часы — положительные, **кратные 0,5**, не больше 24 за одну запись.
7. Деньги — `decimal`, округление до копеек. `double` и `float` для денег не использовать.

8. **Конкурентное редактирование:** если запись изменили после того, как её открыли на редактирование, сохранение завершается понятной ошибкой, а не молча затирает чужие изменения.

API

МЕТОД	ПУТЬ	НАЗНАЧЕНИЕ
GET	/api/time-entries	постраничный список за месяц с фильтрами <code>year</code> , <code>month</code> , <code>employeeId</code> , <code>projectId</code> , <code>page</code> , <code>pageSize</code> ; в ответе — ФИО, шифр проекта, часы, применённая ставка, стоимость
PUT	/api/time-entries	создать запись
POST	/api/time-entries/{id}	изменить запись
DELETE	/api/time-entries/{id}	удалить запись
GET	/api/reports/projects	отчёт по проектам за месяц: <code>year</code> , <code>month</code>
GET	/api/employees /api/projects	справочники для выпадающих списков
POST	/api/periods/close /api/periods/open	закрыть и открыть месяц

Отчёт по проектам за месяц возвращает по каждому проекту, где были трудозатраты: часы, стоимость, бюджет, процент освоения бюджета, признак перерасхода (больше 100 %) и признак риска (больше 80 %), плюс итоговую строку.

ТРЕБОВАНИЯ К РЕАЛИЗАЦИИ

- Отчёт считается **агрегацией на стороне MongoDB**. Выгружать все записи в память и складывать в C# нельзя: считайте, что записей несколько миллионов.
- Список записей — с реальной пагинацией на стороне БД.
- Индексы под используемые запросы созданы явно — скриптом, миграцией или на старте — и кратко обоснованы в `NOTES.md`.

- Ошибки бизнес-правил — это `400` или `409` с машиночитаемым кодом и человекочитаемым текстом на русском, а не `500` и не голый текст исключения.

Интерфейс

ЭКРАН 1 — ТАБЕЛЬ

- фильтры: месяц, сотрудник, проект;
- таблица: дата, сотрудник, проект, часы, ставка, стоимость, комментарий, отметка переработки;
- добавление, редактирование и удаление записи — форма в модальном окне;
- ошибки валидации с сервера видит пользователь, а не только консоль;
- итоги по отфильтрованному списку: часы и стоимость.

ЭКРАН 2 — ОТЧЁТ ПО ПРОЕКТАМ

- выбор месяца;
- таблица: проект, часы, стоимость, бюджет, процент освоения, подсветка перерасхода;
- итоговая строка.

Управление состоянием — любое: `mobx`, `mobx-state-tree`, `redux`, `zustand`, `react-query`. В `NOTES.md` напишите, почему выбрали именно это.

Компонентная библиотека — любая или никакой.

Технические требования

- Backend на `.NET`; можно `.NET 8` — у нас `.NET 5`, но переносимость подхода важнее версии. Контроллеры тонкие, бизнес-логика вынесена (`MediatR` или обычные сервисы — на ваш выбор), валидация входных данных отделена от бизнес-правил.
- MongoDB официальным драйвером. ORM поверх Mongo не использовать.
- **Тесты:** минимум пять юнит-тестов на бизнес-правила — выбор ставки по дате, лимит часов за день, закрытый период, границы периода проекта, округление денег.
- Запуск: `docker compose up` либо README не более чем из пяти шагов. Обязательно — команда или скрипт, наполняющий базу тестовыми данными.

ЧЕГО ДЕЛАТЬ НЕ НУЖНО

Аутентификации, ролей и прав доступа. Красивой вёрстки, адаптивности, тёмной темы, анимаций. Импорта, экспорта, печатных форм, уведомлений. CI/CD, Kubernetes, микросервисов.

Приёмочные проверки

Наполните базу этими данными — это и есть требуемые сид-данные — и убедитесь, что результаты совпадают. Проверять мы будем именно их.

СОТРУДНИКИ

ФИО	ОТДЕЛ	СТАВКИ
Иванов И. И.	Проектный	500 Р/ч с 01.01.2026; 600 Р/ч с 01.03.2026
Петрова А. С.	Проектный	700 Р/ч с 01.02.2026

ПРОЕКТЫ

ШИФР	НАЗВАНИЕ	БЮДЖЕТ	ПЕРИОД
П-001	Реконструкция цеха	20 000 Р	01.01.2026 – 31.03.2026
П-002	Инженерные сети	5 000 Р	с 01.03.2026, без даты окончания

ЗАПИСИ ТАБЕЛЯ

ДАТА	СОТРУДНИК	ПРОЕКТ	ЧАСЫ	ОЖИДАЕМАЯ СТОИМОСТЬ
20.02.2026	Иванов	П-001	8	4 000 Р
05.03.2026	Иванов	П-001	8	4 800 Р — ставка уже 600
05.03.2026	Петрова	П-001	4	2 800 Р
06.03.2026	Петрова	П-002	10	7 000 Р

ПРОЕКТ	ЧАСЫ	СТОИМОСТЬ	БЮДЖЕТ	ОСВОЕНО
П-001	12	7 600 Р	20 000 Р	38 %
П-002	10	7 000 Р	5 000 Р	140 % — перерасход
Итого	22	14 600 Р		

Отчёт за февраль 2026: П-001 — 8 часов, 4 000 Р, освоено 20 %.

СЦЕНАРИИ С ОШИБКАМИ

Каждый должен давать понятное сообщение в интерфейсе, а не «500» и не молчание.

- 1. Запись Петровой на 15.01.2026 — на эту дату у неё ещё нет ставки.
- 2. Запись Иванову 06.03.2026, 20 часов на П-001 — сохраняется, помечается как переработка.
- 3. Следом ещё одна запись Иванову 06.03.2026, 6 часов — суммарно 26 часов за день, отказ.
- 4. Запись на П-002 датой 20.02.2026 — раньше начала проекта, отказ.
- 5. Закрывать февраль 2026 и попробовать изменить запись от 20.02.2026 — отказ.
- 6. Часы или — ошибка валидации прямо в форме.
- 7. Открыть одну и ту же запись в двух вкладках, сохранить в первой, затем во второй — вторая получает внятный отказ.
- 8. Изменить ставку Иванова с 01.03.2026 на 650 Р и перестроить отчёт за март: стоимость записи от 05.03.2026 становится 5 200 Р.

Что прислать

Ссылку на репозиторий — GitHub или GitLab, публичный или доступ по приглашению. В нём:

README.md	как запустить за пять шагов и как загрузить тестовые данные
REVIEW.md	ответ по части 1

NOTES.md	принятые допущения, объяснение выбранных решений, обоснование индексов, что осознанно не доделано и что сделали бы иначе, будь времени вдвое больше
код и тесты	история коммитов — не «одним коммитом»

Дальше — созвон на час: пройдемся по вашему коду, по вашему ревью и по паре вопросов «а как бы вы это масштабировали».

Вопросы по заданию — на почту, с которой вы его получили. Спрашивать можно на любом этапе.